

Sessão 3 - Estatísticas descritivas

Treinamento em R - SEFAZ RS-WB DIME

Thiago Scot

The World Bank | Fevereiro 2025



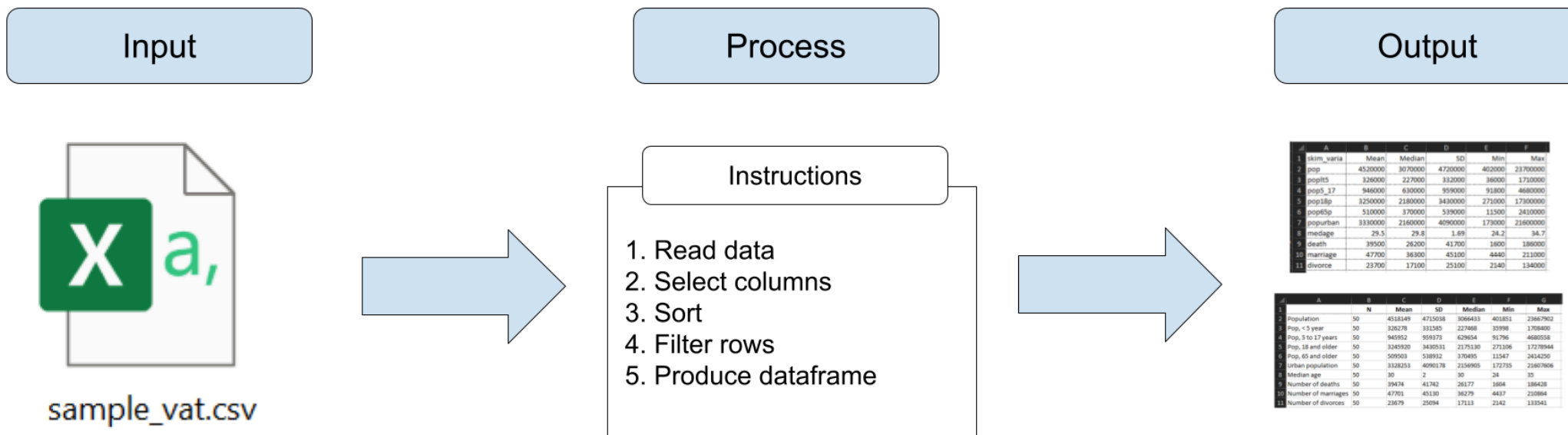
Índice

- Introdução
- Piping
- Estatísticas resumidas rápidas
- Estatísticas resumidas personalizadas
- Exportando tabelas
- Personalizando saídas de tabelas
- Conclusão

Introdução

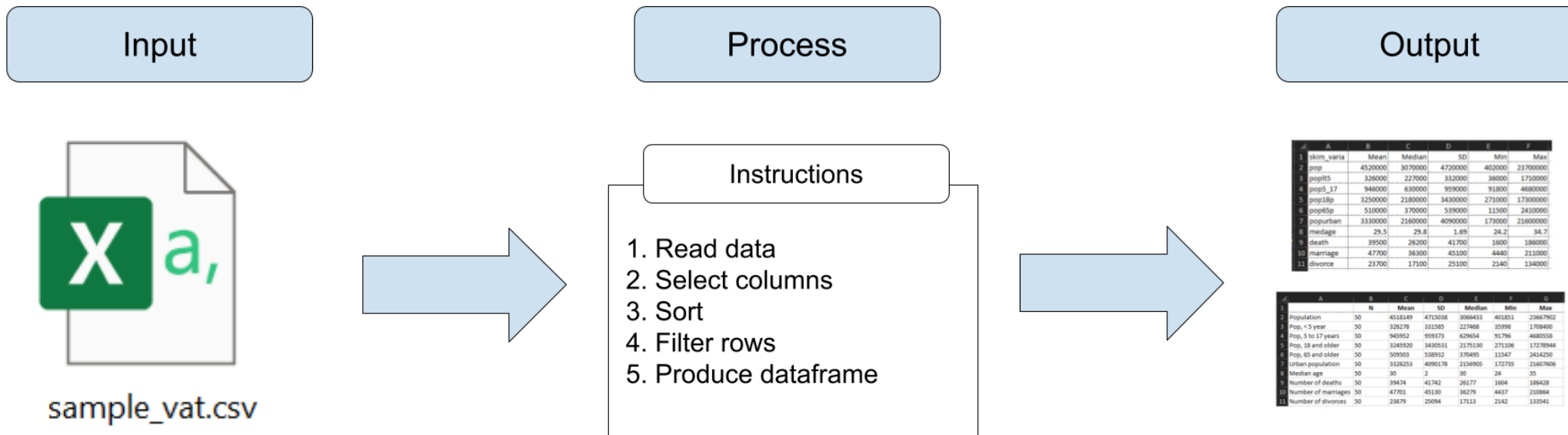
Introdução

- Ontem aprendemos como fazer programação estatística e exportar os resultados em arquivos `.csv`
- No entanto, às vezes precisamos de tabelas mais refinadas do que CSVs simples (e feios)



Introdução

- É disso que trata a sessão de hoje, junto com uma explicação sobre os pipes (*encadeamento*) (`%>%`)



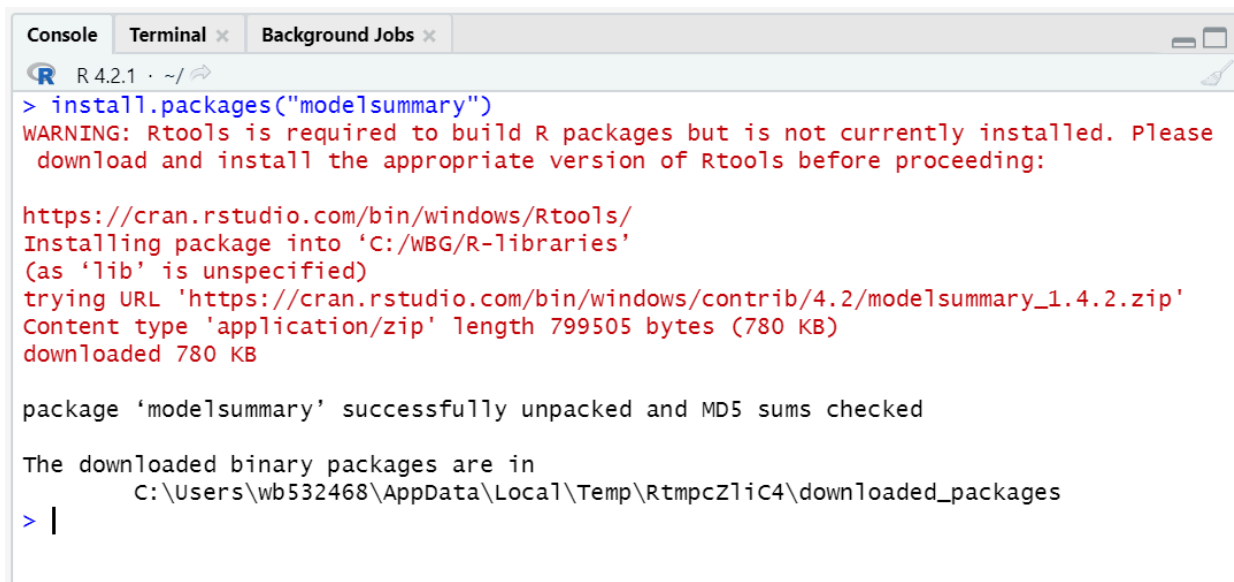
Introdução

Exercício 1a: Obtendo as bibliotecas para a sessão de hoje

Vamos usar duas bibliotecas R nesta sessão: `modelsummary` e `huxtable`.

1. Instale `modelsummary` e `huxtable`:

```
install.packages("modelsummary")  
install.packages("huxtable")
```



```
Console Terminal x Background Jobs x  
R 4.2.1 · ~/   
> install.packages("modelsummary")  
WARNING: Rtools is required to build R packages but is not currently installed. Please  
download and install the appropriate version of Rtools before proceeding:  
  
https://cran.rstudio.com/bin/windows/Rtools/  
Installing package into 'C:/WBG/R-libraries'  
(as 'lib' is unspecified)  
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.2/modelsummary_1.4.2.zip'  
Content type 'application/zip' length 799505 bytes (780 KB)  
downloaded 780 KB  
  
package 'modelsummary' successfully unpacked and MD5 sums checked  
  
The downloaded binary packages are in  
C:\Users\wb532468\AppData\Local\Temp\RtmpcZ1iC4\downloaded_packages  
> |
```

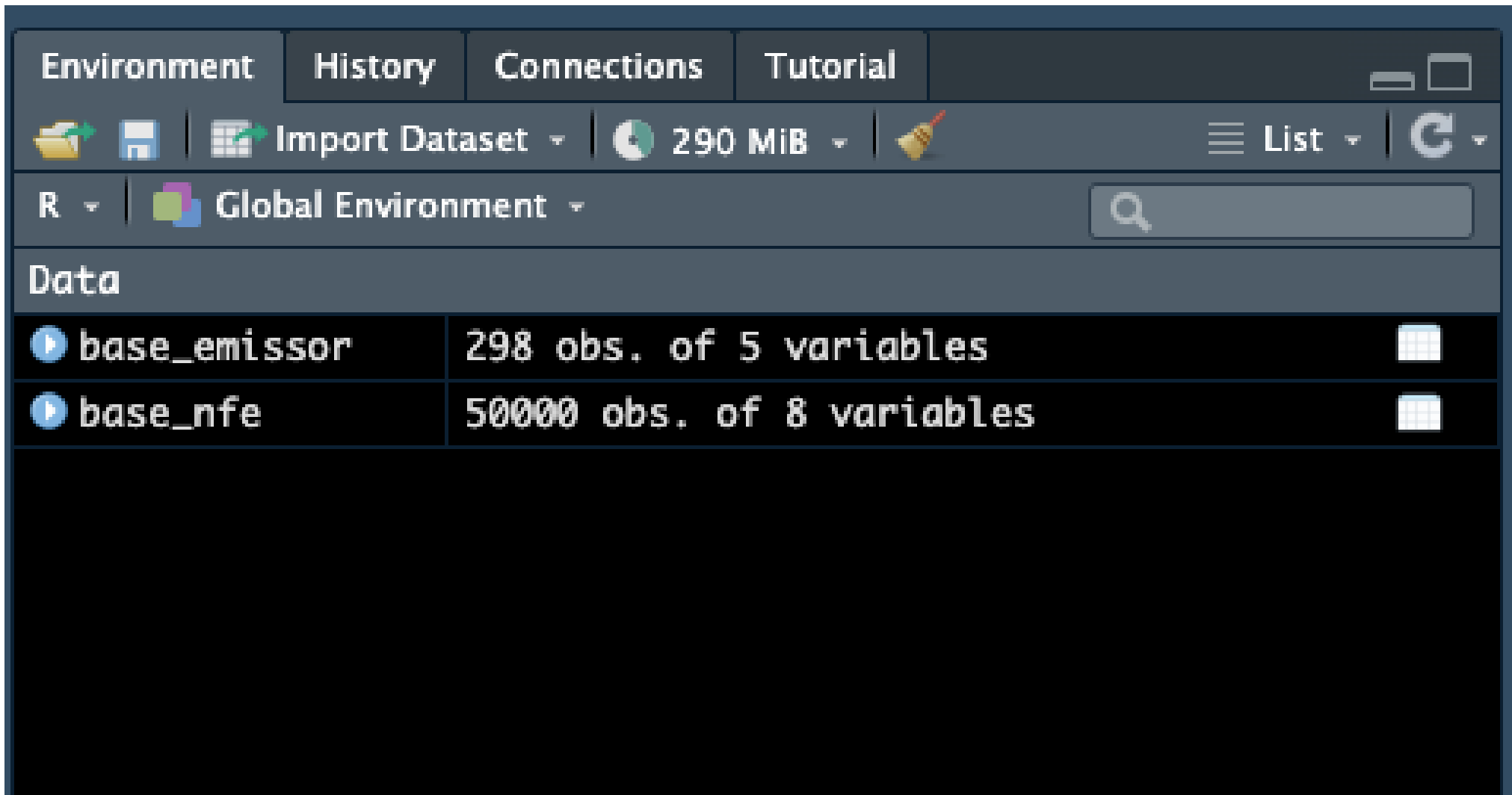
Introdução

Exercício 1b: Carregue os dados que usaremos

```
base_emissor <- read.csv("/Users/tscot/Downloads/Treinamento_SEFAZ/base_emissor.csv")  
base_nfe <- read.csv('/Users/tscot/Downloads/Treinamento_SEFAZ/base_nfe.csv')
```

Introdução

Você deve ter esses dois dataframes carregados no ambiente após isso.

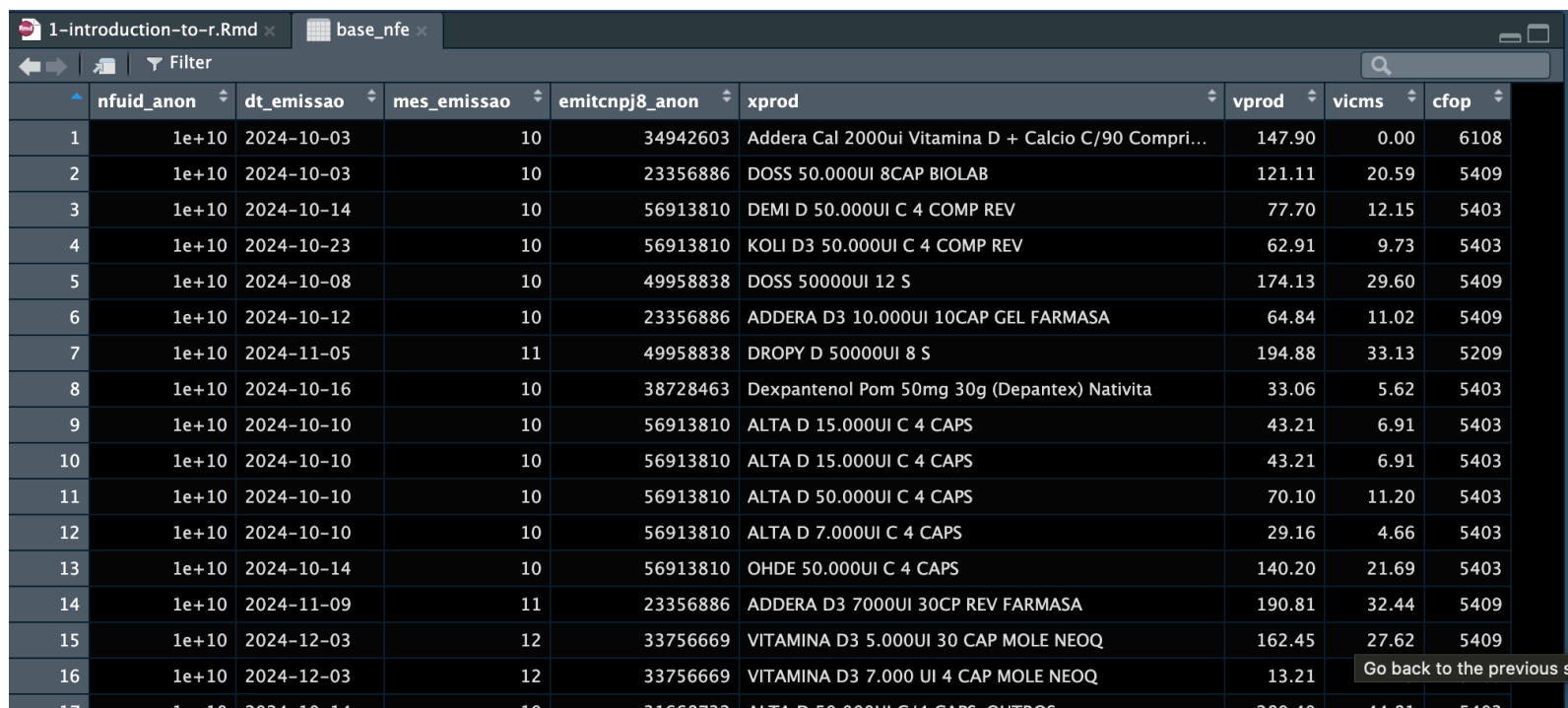


The screenshot shows the RStudio Environment pane. At the top, there are tabs for 'Environment', 'History', 'Connections', and 'Tutorial'. Below the tabs is a toolbar with icons for file operations and a 'List' button. The main area of the pane shows the 'Global Environment' with a search bar. Under the 'Data' section, two dataframes are listed:

Variable	Size
base_emissor	298 obs. of 5 variables
base_nfe	50000 obs. of 8 variables

Introdução

Recapitulação: sempre conheça seus dados!



	nfuid_anon	dt_emissao	mes_emissao	emitcnpj8_anon	xprod	vprod	vicms	cfop
1	1e+10	2024-10-03	10	34942603	Addera Cal 2000ui Vitamina D + Calcio C/90 Compri...	147.90	0.00	6108
2	1e+10	2024-10-03	10	23356886	DOSS 50.000UI 8CAP BIOLAB	121.11	20.59	5409
3	1e+10	2024-10-14	10	56913810	DEMI D 50.000UI C 4 COMP REV	77.70	12.15	5403
4	1e+10	2024-10-23	10	56913810	KOLI D3 50.000UI C 4 COMP REV	62.91	9.73	5403
5	1e+10	2024-10-08	10	49958838	DOSS 50000UI 12 S	174.13	29.60	5409
6	1e+10	2024-10-12	10	23356886	ADDERA D3 10.000UI 10CAP GEL FARMASA	64.84	11.02	5409
7	1e+10	2024-11-05	11	49958838	DROPY D 50000UI 8 S	194.88	33.13	5209
8	1e+10	2024-10-16	10	38728463	Dexpantenol Pom 50mg 30g (Depantex) Nativita	33.06	5.62	5403
9	1e+10	2024-10-10	10	56913810	ALTA D 15.000UI C 4 CAPS	43.21	6.91	5403
10	1e+10	2024-10-10	10	56913810	ALTA D 15.000UI C 4 CAPS	43.21	6.91	5403
11	1e+10	2024-10-10	10	56913810	ALTA D 50.000UI C 4 CAPS	70.10	11.20	5403
12	1e+10	2024-10-10	10	56913810	ALTA D 7.000UI C 4 CAPS	29.16	4.66	5403
13	1e+10	2024-10-14	10	56913810	OHDE 50.000UI C 4 CAPS	140.20	21.69	5403
14	1e+10	2024-11-09	11	23356886	ADDERA D3 7000UI 30CP REV FARMASA	190.81	32.44	5409
15	1e+10	2024-12-03	12	33756669	VITAMINA D3 5.000UI 30 CAP MOLE NEOQ	162.45	27.62	5409
16	1e+10	2024-12-03	12	33756669	VITAMINA D3 7.000 UI 4 CAP MOLE NEOQ	13.21		
17	1e+10	2024-10-14	10	31663733	ALTA D 50.000UI C 4 CAPS OUTROS	200.10	11.01	5403

Piping

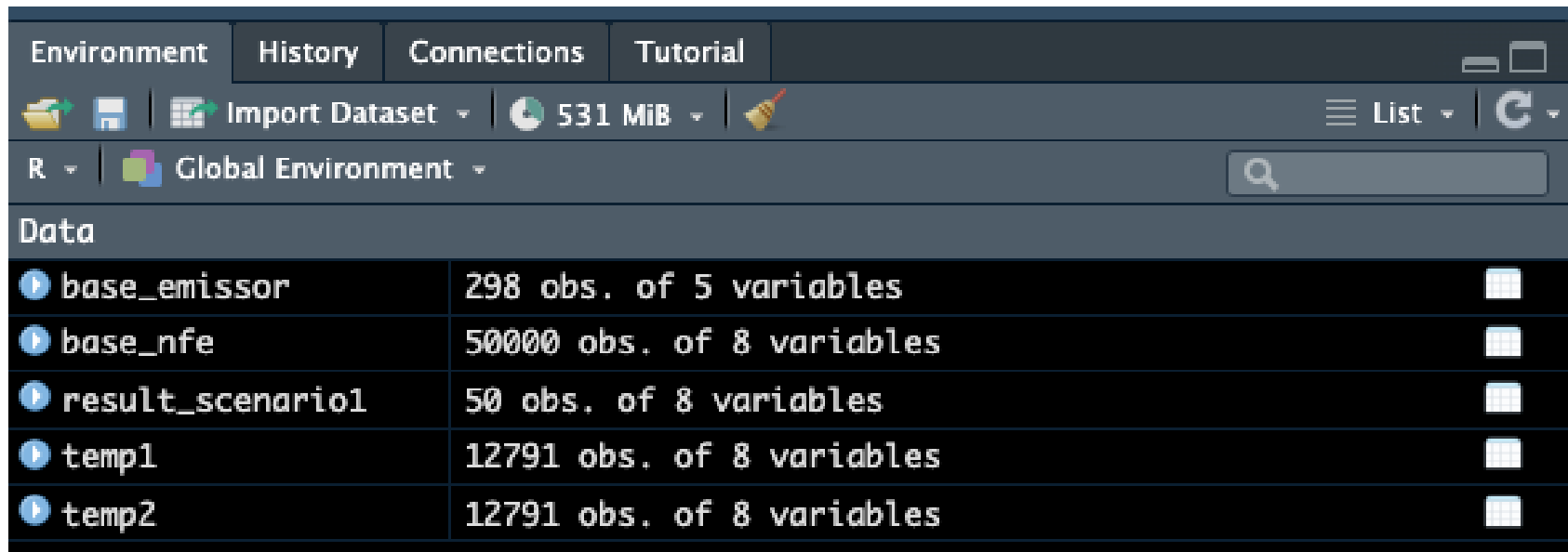
Piping

- Antes de começarmos a produzir saídas mais refinadas, precisamos cobrir o piping (*encadeamento*)
- Você provavelmente se lembra deste código de um dos exercícios de ontem:

```
# Filtrar apenas NFes de dezembro:  
temp1 <- filter(base_nfe, mes_emissao == 12)  
  
# Ordenar resultado anterior por valor, ordem decrescente:  
temp2 <- arrange(temp1, -vprod)  
  
# Manter apenas os 50 primeiros itens após ordenação:  
result_scenario1 <- filter(temp2, row_number() <= 50)
```

Piping

Este código funciona, mas o problema é que é trabalhoso de escrever e nos faz gerar dataframes intermediários desnecessários (`temp1`, `temp2`) que armazenam resultados temporariamente



The screenshot shows the RStudio Environment pane. At the top, there are tabs for 'Environment', 'History', 'Connections', and 'Tutorial'. Below these is a toolbar with icons for file operations and a memory usage indicator showing '531 MiB'. The main area displays the 'Global Environment' with a search bar. A table titled 'Data' lists the objects in the environment:

Object	Description	Icon
base_emissor	298 obs. of 5 variables	Grid icon
base_nfe	50000 obs. of 8 variables	Grid icon
result_scenario1	50 obs. of 8 variables	Grid icon
temp1	12791 obs. of 8 variables	Grid icon
temp2	12791 obs. of 8 variables	Grid icon

Piping

Em vez disso, podemos usar pipes para **passar os resultados de uma função e aplicar uma nova função em cima dela**

```
# Filtrar apenas NFes de dezembro:
temp1 <- filter(base_nfe, mes_emissao == 12)

# Ordenar resultado anterior por valor, ordem decrescente
temp2 <- arrange(temp1, -vprod)

# Manter apenas os 50 primeiros itens após ordenação:
result_scenario1 <- filter(temp2, row_number() <= 50)
```

```
# O mesmo mas com pipes:
result_scenario1 <- base_nfe %>%
  filter(mes_emissao == 12) %>%
  arrange(-vprod) %>%
  filter(row_number() <= 50)
```

Piping

```
# Filtrar apenas NFes de dezembro:
temp1 <- filter(base_nfe, mes_emissao == 12)

# Ordenar resultado anterior por valor, ordem decrescente
temp2 <- arrange(temp1, -vprod)

# Manter apenas os 50 primeiros itens após ordenação:
result_scenariol <- filter(temp2, row_number() <= 50)
```

```
# O mesmo mas com pipes:
result_scenariol <- base_nfe %>%
  filter(mes_emissao == 12) %>%
  arrange(-vprod) %>%
  filter(row_number() <= 50)
```

Existem vários detalhes importantes para notar aqui:

1.- O dataframe resultante `result_scenariol` é **o mesmo em ambos os casos**

Piping

```
# Filtrar apenas NFes de dezembro:
temp1 <- filter(base_nfe, mes_emissao == 12)

# Ordenar resultado anterior por valor, ordem decrescente
temp2 <- arrange(temp1, -vprod)

# Manter apenas os 50 primeiros itens após ordenação:
result_scenariol <- filter(temp2, row_number() <= 50)
```

```
# O mesmo mas com pipes:
result_scenariol <- base_nfe %>%
  filter(mes_emissao == 12) %>%
  arrange(-vprod) %>%
  filter(row_number() <= 50)
```

2.- O nome do dataframe resultante agora é definido na primeira linha desta operação de manipulação de dados. Isso ocorre porque **o R avalia linhas com pipes consecutivos como se fossem uma única linha**

Piping

```
# Filtrar apenas NFes de dezembro:
temp1 <- filter(base_nfe, mes_emissao == 12)

# Ordenar resultado anterior por valor, ordem decrescente
temp2 <- arrange(temp1, -vprod)

# Manter apenas os 50 primeiros itens após ordenação:
result_scenariol <- filter(temp2, row_number() <= 50)
```

```
# O mesmo mas com pipes:
result_scenariol <- base_nfe %>%
  filter(mes_emissao == 12) %>%
  arrange(-vprod) %>%
  filter(row_number() <= 50)
```

3.- Observe que as funções `arrange()` e `filter()` usadas após os pipes agora têm apenas **um argumento em vez de dois**. Isso ocorre porque ao usar pipes, o primeiro argumento é implicitamente o resultado da função antes dos pipes

Exercício 2: filtragem e ordenação revisitadas

1. Aplique a mesma filtragem e ordenação agora com pipes

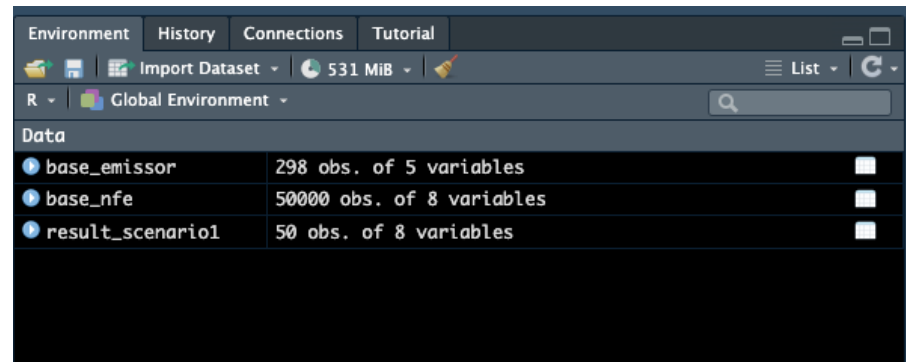
```
result_scenario1 <- base_nfe %>%  
  filter(mes_emissao == 12) %>%  
  arrange(-vprod) %>%  
  filter(row_number() <= 50)
```

- **Atalho:** o RStudio reconhece o atalho `Ctrl+Shift+M` para pipe (`%>%`)

Piping

Agora não teremos nenhum resultado intermediário irritante armazenado em nosso ambiente!

```
# O mesmo mas com pipes:  
result_scenario1 <- base_nfe %>%  
  filter(mes_emissao == 12) %>%  
  arrange(-vprod) %>%  
  filter(row_number() <= 50)
```



The screenshot shows the RStudio Environment pane. At the top, there are tabs for 'Environment', 'History', 'Connections', and 'Tutorial'. Below these, there's a toolbar with icons for 'Import Dataset', memory usage '531 MiB', and a 'List' button. The main area shows the 'Global Environment' with a search bar. Under the 'Data' section, three objects are listed:

Object	Description	Icon
base_emissor	298 obs. of 5 variables	📄
base_nfe	50000 obs. of 8 variables	📄
result_scenario1	50 obs. of 8 variables	📄

Sempre use pipes!

Agora que você conhece o poder dos pipes, use-os sabiamente!

- Lembre-se que os pipes fazem parte da biblioteca `dplyr`, você precisa carregá-la antes de usá-los
- Os pipes também melhoram drasticamente a clareza do código
- Muitos programadores R usam pipes e exemplos da internet assumem que você os conhece
- **Usaremos pipes agora nos próximos exemplos e exercícios do resto deste treinamento**



Estatísticas resumidas rápidas

Estatísticas resumidas rápidas

Aprendemos ontem como produzir dataframes com resultados e exportá-los.

Mas e se você quiser...?

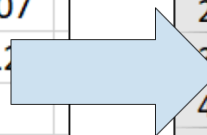
- ...exportar resultados em um formato diferente (exemplo: Excel)
- ...personalizar ainda mais quais linhas e colunas exibir em um resultado
- ...formatar os resultados que você exporta

Estatísticas resumidas rápidas

Você precisará de `modelsummary` e `huxtable` para isso

- Essas bibliotecas permitem exportar resultados de forma personalizada
- Escolhemos uma combinação de ambas porque juntas exportam uma grande variedade de tipos de saída e permitem personalização detalhada das saídas

	A	B	C	D	E	F
1		mean	sd	min	median	max
2	modifie	#####	#####	19832	5008712	1.2E+07
3	taxperio	201907	3	201901	201907	201911
4	age	14	8	1	13	30
5	income	3284	8242	0	907	139395
6	vat_liabi	591	1484	0	163	25091



	A	B	C	D	E
1		Unique (#)	Missing (%)	Mean	SD
2	modified_id	984	0	5448915	3758602
3	taxperiod	12	0	201907	3
4	age	30	0	14	8
5	income	721	0	3284	8242
6	vat_liability	721	0	591	1484

Estatísticas resumidas rápidas

Começaremos apresentando a função `datasummary_skim()` do `modelsummary`

```
datasummary_skim(data, output, ...)
```

- **data:** o conjunto de dados a ser resumido, o único argumento obrigatório
- **output:** o tipo de saída desejada
- **...:** opções adicionais permitem personalização de formatação, como incluir notas e títulos

Por exemplo:

```
datasummary_skim(  
  data,  
  output = "default",  
  type = "numeric",  
  title = NULL,  
  notes = NULL,  
  ...  
)
```

Estatísticas resumidas rápidas

Exercício 3: Calcular estatísticas resumidas rápidas

1. Carregue `modelsummary` com `library(modelsummary)`
2. Use `datasummary_skim()` para criar uma tabela de estatísticas descritivas para `base_nfe`

Estatísticas resumidas rápidas

Você deve estar vendo este resultado no painel inferior direito do RStudio.

```
##
## +-----+-----+-----+-----+-----+-----+-----+-----+-----+
## |           | Unique | Missing Pct. | Mean           | SD           | Min           | Median         | Max           |
## +=====+=====+=====+=====+=====+=====+=====+=====+=====+
## | nfuid_anon  | 35168  | 0             | 1.000002e+10 | 10137.3      | 10000000000.0 | 1.000002e+10 | 1.000004e+10 |
## +-----+-----+-----+-----+-----+-----+-----+-----+-----+
## | mes_emissao | 3      | 0             | 1.080000e+01 | 0.8          | 10.0          | 1.100000e+01 | 1.200000e+01 |
## +-----+-----+-----+-----+-----+-----+-----+-----+-----+
## | emitcnpj8_anon | 298    | 0             | 3.866839e+07 | 16480338.8   | 20076758.0    | 3.375667e+07 | 8.946785e+07 |
## +-----+-----+-----+-----+-----+-----+-----+-----+-----+
## | vprod       | 8707   | 0             | 1.537000e+02 | 2213.6       | 0.0           | 4.220000e+01 | 1.785175e+05 |
## +-----+-----+-----+-----+-----+-----+-----+-----+-----+
## | vicms       | 3625   | 0             | 1.460000e+01 | 128.1        | 0.0           | 5.200000e+00 | 1.180940e+04 |
## +-----+-----+-----+-----+-----+-----+-----+-----+-----+
## | cfop        | 31     | 0             | 5.578800e+03 | 352.6        | 5102.0        | 5.409000e+03 | 6.949000e+03 |
## +-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Estatísticas resumidas rápidas

- A maioria das funções do `modelsummary` resume apenas variáveis numéricas por padrão
- Para resumir variáveis categóricas, use o argumento `type = "categorical"`

```
datasummary_skim(base_emissor, type = "categorical")
```

Estatísticas resumidas rápidas

Warning: These variables were omitted because they include more than 50 levels:
emitxnomefantasia.

```
##
```

		N	%
emituf	MG	5	1.7
	PR	4	1.3
	RS	281	94.3
	SC	1	0.3
	SP	7	2.3
emitcategoria	GERAL	198	66.4
	SEM CADASTRO	1	0.3
	SIMPLES NACIONAL	99	33.2

Estatísticas resumidas rápidas

- `datasummary_skim()` é conveniente porque é rápido, fácil e mostra muitas informações

```
## Warning: These variables were omitted because they include more than 50 levels:
```

```
## emitxnomefantasia.
```

```
##
```

```
## +-----+-----+-----+-----+
## |          |          | N   | %   |
## +=====+=====+=====+=====+
```

```
## | emituf      | MG          | 5   | 1.7 |
## +-----+-----+-----+-----+
```

```
## |          | PR          | 4   | 1.3 |
## +-----+-----+-----+-----+
```

```
## |          | RS          | 281 | 94.3 |
## +-----+-----+-----+-----+
```

```
## |          | SC          | 1   | 0.3 |
## +-----+-----+-----+-----+
```

```
## |          | SP          | 7   | 2.3 |
## +-----+-----+-----+-----+
```

```
## |          | SEM CADASTRO | 1   | 0.3 |
```

```
## | emitcategoria | GERAL          | 198 | 66.4 |
## +-----+-----+-----+-----+
```

```
## |          | SEM CADASTRO | 1   | 0.3 |
```

```
## |          | SEM CADASTRO | 1   | 0.3 |
```

```
## |          | SEM CADASTRO | 1   | 0.3 |
```

```
## |          | SEM CADASTRO | 1   | 0.3 |
```

```
## |          | SEM CADASTRO | 1   | 0.3 |
```

```
## |          | SEM CADASTRO | 1   | 0.3 |
```

```
## |          | SEM CADASTRO | 1   | 0.3 |
```

```
## |          | SEM CADASTRO | 1   | 0.3 |
```

Estatísticas resumidas personalizadas

Estatísticas resumidas personalizadas

`datasummary()` é muito similar a `data_summary_skim()`. A única diferença é que requer um **argumento de fórmula**.

`datasummary(formula, data, output)`

- **formula:** uma fórmula de dois lados para descrever a tabela como: linhas ~ colunas
- **data:** o conjunto de dados a ser resumido
- **output:** o tipo de saída desejada
- **...**: opções adicionais permitem personalização de formatação

```
datasummary(  
  var1 + var2 + var3 ~ stat1 + stat2 + stat3 + stat4,  
  data = data  
)
```

Estatísticas resumidas personalizadas

Exercício 4:

Crie uma tabela de estatísticas resumidas mostrando o número de observações, média, desvio padrão, mínimo e máximo para as variáveis `vprod`, `vicms` do dataframe `base_nfe`

1. Use `datasummary()` para isso:

```
datasummary(  
  vprod + vicms ~ N + Mean + SD + Min + Max,  
  base_nfe  
)
```

Estatísticas resumidas personalizadas

```
##
## +-----+-----+-----+-----+-----+-----+
## |      | N      | Mean   | SD      | Min    | Max      |
## +=====+=====+=====+=====+=====+=====+
## | vprod | 50000  | 153.70 | 2213.65 | 0.00   | 178517.50 |
## +-----+-----+-----+-----+-----+-----+
## | vicms | 50000  | 14.64  | 128.15  | 0.00   | 11809.44  |
## +-----+-----+-----+-----+-----+-----+
```


Estatísticas resumidas personalizadas

```
datasummary(  
  vprod + vicms ~ N + Mean + SD + Min + Max,  # esta é a fórmula  
  base_nfe                                     # estes são os dados  
)
```

Algumas observações:

- Os argumentos **formula** e **data** são obrigatórios para `datasummary()`
- Todos os outros argumentos são opcionais (como `title = *algum-título*`, para adicionar um título à tabela)
- A fórmula deve sempre ser definida como: linhas ~ colunas
- As linhas e colunas na fórmula são separadas por um sinal de mais (+)

Estatísticas resumidas personalizadas

```
datasummary(  
  vprod + vicms ~ N + Mean + SD + Min + Max,  # esta é a fórmula  
  base_nfe                                     # estes são os dados  
)
```

Nesse exercício utilizamos as estatísticas número de observações (**N**), média (**mean**), desvio-padrão (**SD**), mínimo (**Min**) e máximo (**Max**). Outras estatísticas incluem

Estatística	Keyword
Mediana	Median
Percentile 25	P25
percentile 75	P75
Em geral: percentil XX	PXX
Histograma pequeno	Histogram

Exportando tabelas

Exportando tabelas

Lembre-se que tanto `datasummary_skim()` quanto `datasummary()` têm um argumento opcional chamado *output*? Podemos usá-lo para especificar um caminho de arquivo para um arquivo de saída.

Por exemplo:

```
datasummary_skim(base_nfe,  
                  output = "quick_stats.docx")
```

Irá exportar o resultado para a pasta `Documentos` (no Windows) em um arquivo Word chamado `quick_stats.docx`

Exportando tabelas

O tipo de arquivo da saída é ditado pela extensão do arquivo. Por exemplo:

Nome do arquivo	Extensão	Tipo de saída
"quick_stats.docx"	<code>.docx</code>	Word
"quick_stats.pptx"	<code>.pptx</code>	Power Point
"quick_stats.html"	<code>.html</code>	HTML (para abrir em um navegador)
"quick_stats.tex"	<code>.tex</code>	Latex
"quick_stats.md"	<code>.md</code>	Markdown

Notou que está faltando Excel?

Exportando tabelas

Isso é porque as funções do `modelsummary` não podem exportar para Excel

- No entanto, podemos usar a biblioteca `huxtable` como intermediária para transformar resultados de funções `modelsummary` em arquivos Excel
- `huxtable` é um pacote para exportar tabelas em geral que permite **personalizar a saída que você está exportando**
- Vamos aprender como usá-lo no próximo exercício

Exportando tabelas

Exercício 5: Exportar uma tabela para Excel

1. Carregue `huxtable` com `library(huxtable)`
2. Execute o seguinte código para exportar o resultado de `datasummary_skim()` para Excel:

```
# Armazene a tabela em um novo objeto  
stats_table <- datasummary_skim(base_nfe, output = "huxtable")  
  
# Exporte este novo objeto para Excel com quick_xlsx()  
quick_xlsx(stats_table, file = "quick_stats.xlsx")
```

Exportando tabelas

Agora o resultado deve aparecer na pasta **Documents**

Name	Date modified	Type	Size
.Rproj.user	9/19/2023 2:37 AM	File folder	
1-introduction-to-r_cache	9/19/2023 3:30 AM	File folder	
2-data-wrangling_cache	9/19/2023 3:46 PM	File folder	
3-descriptive-statistics_cache	9/20/2023 7:01 PM	File folder	
4-data-visualization_cache	9/20/2023 11:23 PM	File folder	
4-data-visualization_files	9/21/2023 12:02 AM	File folder	
data	9/20/2023 11:51 PM	File folder	
img	9/20/2023 11:10 PM	File folder	
libs	9/21/2023 5:55 AM	File folder	
1-introduction-to-r.pdf	9/20/2023 9:47 AM	Adobe Acrobat Docu...	4,090 KB
2-data-wrangling.pdf	9/20/2023 9:48 AM	Adobe Acrobat Docu...	5,163 KB
1-introduction-to-r.html	9/20/2023 1:34 AM	Chrome HTML Docu...	30 KB
2-data-wrangling.html	9/20/2023 5:44 AM	Chrome HTML Docu...	33 KB
3-descriptive-statistics.html	9/21/2023 5:55 AM	Chrome HTML Docu...	66 KB
4-data-visualization.html	9/21/2023 5:15 AM	Chrome HTML Docu...	29 KB
df_tbilisi_50.csv	9/20/2023 5:12 AM	Microsoft Excel Com...	2 KB
total_income.csv	9/20/2023 5:11 AM	Microsoft Excel Com...	1 KB
quick_stats.xlsx	9/21/2023 6:07 AM	Microsoft Excel Work...	7 KB
quick_stats.docx	9/21/2023 5:57 AM	Microsoft Word Doc...	13 KB

Exportando tabelas

E voce pode abrir no Excel e ajustar caso queira (**mas lembre-se dos objetivos de programação: reproducibilidade é afetada se fazemos ajustes manuais no Excel**)

	A	B	C	D	E	F	G	H	I	J	K
1		Unique (#)	Missing (%)	Mean	SD	Min	Median	Max			
2	modified_id	984	0	5448915.1	3758602.4	19832	5008712	12296912			
3	taxperiod	12	0	201906.7	3.4	201901	201907	201912			
4	age	30	0	14	8.4	1	13	30			
5	income	721	0	3283.9	8242.4	0	906.8	139394.5			
6	vat_liability	721	0	591.1	1483.6	0	163.2	25091			
7											
8											
9											
10											
11											
12											
13											
14											
15											
16											

Exportando tabelas

```
# Armazene a tabela em um novo objeto  
stats_table <- datasummary_skim(base_nfe, output = "huxtable")  
  
# Exporte este novo objeto para Excel com quick_xlsx()  
quick_xlsx(stats_table, file = "quick_stats.xlsx")
```

Alguns comentários sobre esse código:

- `quick_xlsx()` é uma função do pacote `huxtable`. O primeiro argumento é o objeto que queremos exports, e o segundo o nome do arquivo. Poderíamos também usar um caminho (`path`) pra indicar onde salvar o arquivo
- Note que utilizamos o argumento `output = "huxtable"` em `datasummary_skim()`. Isso indica ao R que o resultado deve ser um objeto que depois podemos utilizar com funções do `huxtable`, como `quick_xlsx()`

Personalizando saídas de tabelas

Personalizando saídas de tabelas

O código abaixo mostra como a tabela `stats_table` pode ser formatada:

```
# Começamos com stats_table:
stats_table %>%
  # Use primeira linha como cabeçalho da tabela
  set_header_rows(1, TRUE) %>%
  # Use primeira coluna como cabeçalho de linha
  set_header_cols(1, TRUE) %>%
  # Não arredonde números grandes
  set_number_format(everywhere, 2:ncol(.), "%9.0f") %>%
  # Centralize células na primeira linha
  set_align(1, everywhere, "center") %>%
  # Defina um tema para formatação rápida
  theme_basic()
```

	Unique	Missing Pct.	Mean	SD	
nfuid_anon	35168	0	10000020000	10137	1
mes_emissao	3	0	11	1	1
emitcnpj8_anon	298	0	38668390	16480339	2
vprod	8707	0	154	2214	0
vicms	3625	0	15	128	0
cfop	31	0	5579	353	5

Personalizando saídas de tabelas

Exercício 6: Exportar uma tabela personalizada para Excel

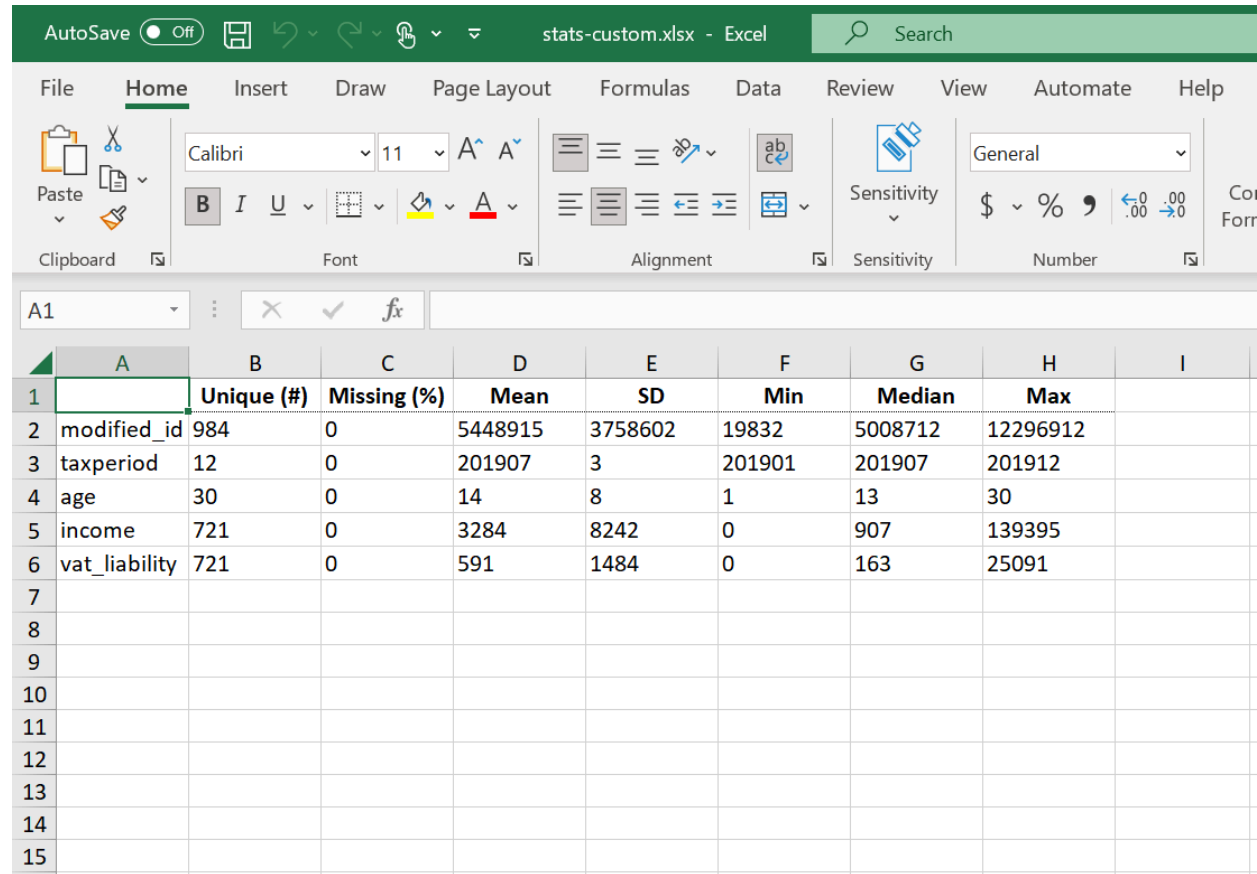
1.- Personalize `stats_table` em um novo objeto chamado `stats_table_custom`

```
stats_table_custom <- stats_table %>%  
  # Use primeira linha como cabeçalho da tabela  
  set_header_rows(1, TRUE) %>%  
  # Use primeira coluna como cabeçalho de linha  
  set_header_cols(1, TRUE) %>%  
  # Não arredonde números grandes  
  set_number_format(everywhere, 2:ncol(.), "%9.0f") %>%  
  # Centralize células na primeira linha  
  set_align(1, everywhere, "center") %>%  
  # Defina um tema para formatação rápida  
  theme_basic()
```

2.- Exporte `stats_table_custom` para um arquivo chamado `stats-custom.xlsx` com `quick_xlsx()`

```
quick_xlsx(  
  stats_table_custom,  
  file = "stats-custom.xlsx"  
)
```

Customizing table outputs



The screenshot shows the Microsoft Excel interface with the 'Home' tab selected. The ribbon includes options for Clipboard, Font, Alignment, Sensitivity, and Number. The active cell is A1. The following table is displayed in the worksheet:

	A	B	C	D	E	F	G	H	I
1		Unique (#)	Missing (%)	Mean	SD	Min	Median	Max	
2	modified_id	984	0	5448915	3758602	19832	5008712	12296912	
3	taxperiod	12	0	201907	3	201901	201907	201912	
4	age	30	0	14	8	1	13	30	
5	income	721	0	3284	8242	0	907	139395	
6	vat_liability	721	0	591	1484	0	163	25091	
7									
8									
9									
10									
11									
12									
13									
14									
15									

Customizing table outputs

Note que primeiro guardamos o resultado em um objeto...

```
stats_table_custom <- stats_table %>% # <---- here
  set_header_rows(1, TRUE) %>%
  set_header_cols(1, TRUE) %>%
  set_number_format(everywhere, 2:ncol(.), "%9.0f") %>%
  set_align(1, everywhere, "center") %>%
  theme_basic()
```

...que depois pode ser utilizado como argumento na função `quick_xlsx()`

```
quick_xlsx(
  stats_table_custom,
  file = "stats-custom.xlsx"
)
```

Customizing table outputs

Before:

Excel interface showing a table with 8 columns: Unique (#), Missing (%), Mean, SD, Min, Median, Max. The table is located in the range A1:H6.

	A	B	C	D	E	F	G	H	I	J	K
1		Unique (#)	Missing (%)	Mean	SD	Min	Median	Max			
2	modified_id	984	0	5448915.1	3758602.4	19832	5008712	12296912			
3	taxperiod	12	0	201906.7	3.4	201901	201907	201912			
4	age	30	0	14	8.4	1	13	30			
5	income	721	0	3283.9	8242.4	0	906.8	139394.5			
6	vat_liability	721	0	591.1	1483.6	0	163.2	25091			
7											
8											
9											
10											
11											
12											
13											
14											
15											
16											

After:

Excel interface showing a table with 8 columns: Unique (#), Missing (%), Mean, SD, Min, Median, Max. The table is located in the range A1:H6. The table is formatted with bold headers and a light blue background for the first column.

	A	B	C	D	E	F	G	H	I
1		Unique (#)	Missing (%)	Mean	SD	Min	Median	Max	
2	modified_id	984	0	5448915	3758602	19832	5008712	12296912	
3	taxperiod	12	0	201907	3	201901	201907	201912	
4	age	30	0	14	8	1	13	30	
5	income	721	0	3284	8242	0	907	139395	
6	vat_liability	721	0	591	1484	0	163	25091	
7									
8									
9									
10									
11									
12									
13									
14									
15									

Personalizando saídas de tabelas

Usamos `theme_basic()` para dar um tema minimalista e básico à tabela. Outros temas disponíveis são:

theme_plain

Type	Price	Sugar content
Strawberry	1.90	40.00%
Raspberry	2.10	35.00%
Plum	1.80	50.00%

theme_basic

Type	Price	Sugar content
Strawberry	1.90	40.00%
Raspberry	2.10	35.00%
Plum	1.80	50.00%

theme_compact

Type	Price	Sugar content
Strawberry	1.90	40.00%
Raspberry	2.10	35.00%
Plum	1.80	50.00%

theme_article

Type	Price	Sugar content
Strawberry	1.90	40.00%
Raspberry	2.10	35.00%
Plum	1.80	50.00%

theme_bright

Type	Price	Sugar content
Strawberry	1.90	40.00%
Raspberry	2.10	35.00%
Plum	1.80	50.00%

theme_grey

Type	Price	Sugar content
Strawberry	1.90	40.00%
Raspberry	2.10	35.00%
Plum	1.80	50.00%

theme_blue

Type	Price	Sugar content
Strawberry	1.90	40.00%
Raspberry	2.10	35.00%
Plum	1.80	50.00%

theme_green

Type	Price	Sugar content
Strawberry	1.90	40.00%
Raspberry	2.10	35.00%
Plum	1.80	50.00%

theme_mondrian

Type	Price	Sugar content
Strawberry	1.90	40.00%
Raspberry	2.10	35.00%
Plum	1.80	50.00%

theme_orange

Type	Price	Sugar content
Strawberry	1.90	40.00%
Raspberry	2.10	35.00%
Plum	1.80	50.00%

theme_stripped

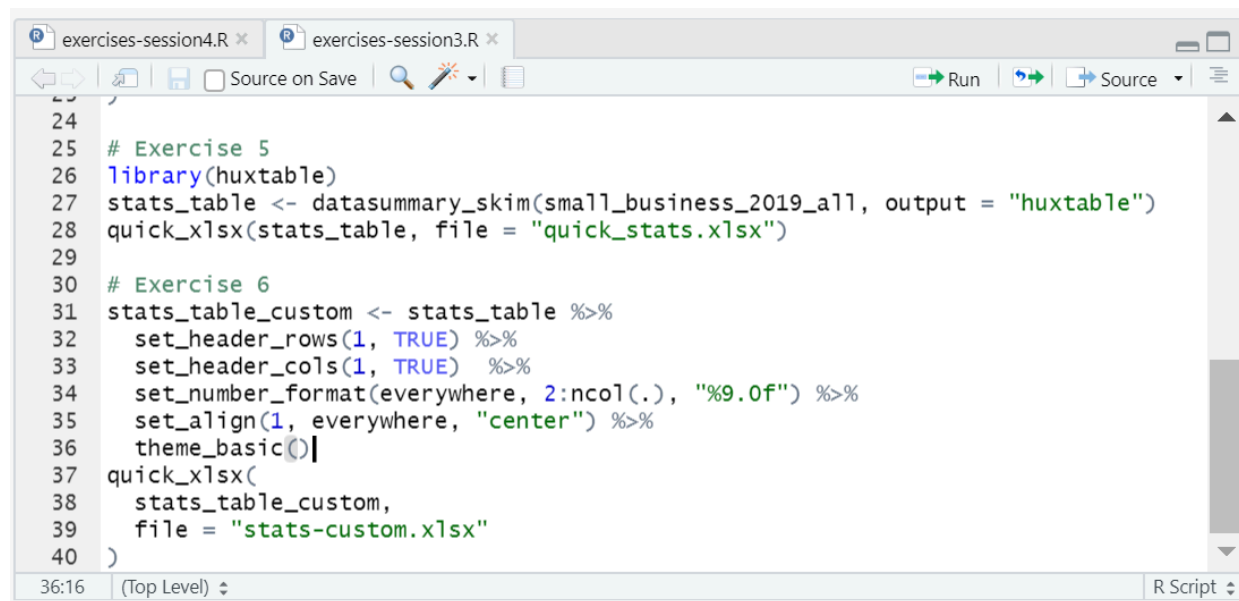
Type	Price	Sugar content
Strawberry	1.90	40.00%
Raspberry	2.10	35.00%
Plum	1.80	50.00%

Conclusão

Conclusão

Salve seu trabalho!

Clique no disquete para salvar o script que você escreveu nesta sessão.



```
24  
25 # Exercise 5  
26 library(huxtable)  
27 stats_table <- datasummary_skim(small_business_2019_all, output = "huxtable")  
28 quick_xlsx(stats_table, file = "quick_stats.xlsx")  
29  
30 # Exercise 6  
31 stats_table_custom <- stats_table %>%  
32   set_header_rows(1, TRUE) %>%  
33   set_header_cols(1, TRUE) %>%  
34   set_number_format(everywhere, 2:ncol(.), "%9.0f") %>%  
35   set_align(1, everywhere, "center") %>%  
36   theme_basic()|  
37 quick_xlsx(  
38   stats_table_custom,  
39   file = "stats-custom.xlsx"  
40 )
```

Conclusão

O que mais está disponível?

- Esta foi uma breve visão geral de como `modelsummary` e `huxtable` trabalham juntos para produzir saídas de tabela com aparência profissional em R
- Outras opções de formatação são: (todas do `huxtable`)

Formatação	Comando
Exportar em novas abas do Excel em vez de novos arquivos	<code>as_workbook()</code>
Mudar nomes de linhas	<code>add_rownames()</code>
Mudar nomes de colunas	<code>add_colnames()</code>
Células em negrito	<code>set_bold()</code>
Células em itálico	<code>set_italic()</code>
Tamanho da fonte da célula	<code>font_size()</code>
Cor da célula	<code>background_color()</code>

Conclusão

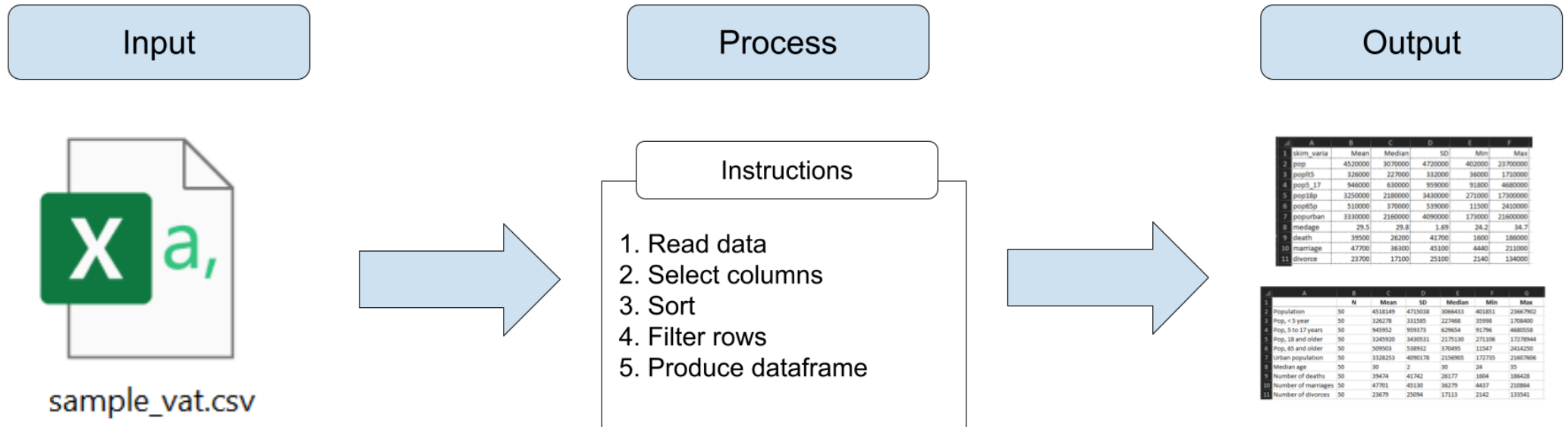
O que mais está disponível?

Mais informações estão explicadas na documentação das bibliotecas:

- Documentação do `modelsummary`: <https://modelsummary.com/index.html>
- Documentação do `huxtable`: <https://hughjonesd.github.io/huxtable/>

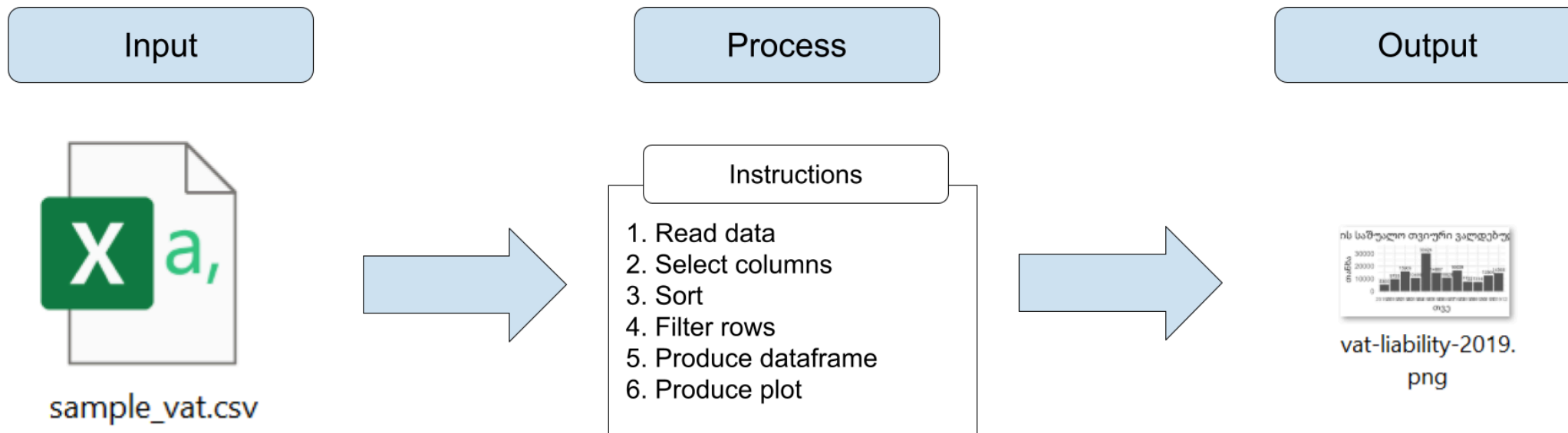
Conclusão

Esta sessão



Conclusão

Próxima sessão (última)



Obrigado!
